

Algorithm Design and Analysis

Lecturer: Nguyễn Thị Hồng Minh
Trần Bá Tuấn - Đặng Trung Du

HUS - HKII,
2024-2025

Assignment 2

§ Algorithm Analysis §

Phần 1: Mục tiêu

- Sinh viên cần nắm được các kiến thức cơ bản về thuật toán và phân tích thuật toán.
- Sinh viên hiểu được cách đánh giá độ phức tạp của thuật toán và mối quan hệ theo ký pháp tiệm cận.
- Sinh viên nắm rõ cách chứng minh tính đúng đắn của thuật toán.

Phần 2: Thực hành

(1) Quy cách nộp bài

- Nộp bài bằng file văn bản hoặc ảnh chụp scan thành PDF nếu bài làm viết tay ra giấy (khuyến khích sinh viên thực hiện trình bày bài làm bằng Latex).
- Chương trình viết bằng ngôn ngữ Java hoặc Python. Mã nguồn chương trình, file dữ liệu (nếu có) được đặt trong cùng thư mục, kèm theo hướng dẫn cách thực hiện chương trình nếu cần.
- Tất cả các file liên quan tới bài tập để trong thư mục tên Hw2_MaSinhVien_Hovaten, thư mục được nén thành file.zip cùng tên thư mục.
- Khi hết hạn nộp bài, các bài làm sẽ công bố để thực hiện chấm chéo bài tập.
- Sinh viên không nộp bài sẽ nhận điểm 0 bài tập tuần.
- Sinh viên CÓ GIAN LẬN trong nộp bài tập sẽ bị ĐÌNH CHỈ môn học (điểm 0 cho tất cả các điểm thành phần).

(2) LUYỆN TẬP TẠI LỚP

- Trình bày sơ lược về độ phức tạp của thuật toán, mối quan hệ giữa độ phức tạp thuật toán theo ký pháp tiệm cận và các loại ký pháp tiệm cận cho sinh viên.

Ví dụ 1. *Tìm giới hạn trên (upper bound) cho:*

a) $f(n) = 4n + 5$.

b) $f(n) = n^2 + 2$.

c) $f(n) = n^4 + 100n^2 + 50$.

d) $f(n) = 2n^3 - 2n^2$.

d) $f(n) = 508$.

Câu hỏi chung từ VD2 đến VD8 là: Xác định độ phức tạp của thuật toán là gì?

Ví dụ 2. *Tính $S = \frac{n * (n - 1)}{2}$.*

Ví dụ 3.

```
1 for i in range(0, n):
2     print('Current Number', i)
```

Ví dụ 4.

```
1 for i in range(0, n):
2     print('Current Number', i)
3     break
```

Ví dụ 5.

```
1 def function(n):
2     i = 1
3     while i <= n:
4         i = i * 2
5         print(i)
6 function(100)
```

Ví dụ 6.

```
1 for i in range(0, n):
2     for j in range(0, n):
3         print("Value of i, j", i, j)
```

Ví dụ 7.

```
1 public void function(int n){
2     int i, j, k, count = 0;
3     for(i = n/2; i <= n; i++)
4         for(j = 1; j + n/2 <= n; j++)
5             for(k = 1; k <= n; k = k * 2)
6                 count++;
7 }
```

Ví dụ 8.

```

1 public void function(int n) {
2     int i, j, k, count = 0;
3     for(i = n/2; i <= n; i++)
4         for(j = 1; j <= n; j = 2 * j)
5             for(k = 1; k <= n; k = k * 2)
6                 count++;
7 }

```

Ví dụ 9. Độ phức tạp của thuật toán là gì?

- a) $T(n) = n \log n + 3n + 2$
- b) $T(n) = n \log(n!) + 5n^2 + 6$
- c) $T(n) = 100n + 10n^2$
- d) $T(n) = 50n \log n + n^3 + 100n$
- e) $T(n) = 10n \log n + n(\log n)^2$

Ví dụ 10. Độ phức tạp của thuật toán các đoạn code dưới đây là gì?

a)

```

1 // Returns the sum of the integers in given array.
2 public static int example1(int[] arr) {
3     int n = arr.length, total = 0;
4     for (int j=0; j < n; j++) // loop from 0 to n-1
5         total += arr[j];
6     return total;
7 }

```

b)

```

1 // Returns the sum of the integers with even index in given
  ↪ array.
2 public static int example2(int[] arr) {
3     int n = arr.length, total = 0;
4     for (int j=0; j < n; j += 2) // note the increment of 2
5         total += arr[j];
6     return total;
7 }

```

c)

```

1 // Returns the sum of the prefix sums of given array.
2 public static int example3(int[] arr) {
3     int n = arr.length, total = 0;
4     for (int j=0; j < n; j++) // loop from 0 to n-1
5         for (int k=0; k <= j; k++) // loop from 0 to j
6             total += arr[j];
7     return total;
8 }

```

d)

```

1 // Returns the sum of the prefix sums of given array.
2 public static int example4(int[] arr) {
3     int n = arr.length, prefix = 0, total = 0;
4     for (int j=0; j < n; j++) { // loop from 0 to n-1
5         prefix += arr[j];
6         total += prefix;
7     }
8     return total;
9 }

```

e)

```

1 // Returns the number of times second array stores sum of
  ↪ prefix sums from first.
2 public static int example5(int[] first, int[] second) {
3     // assume equal-length arrays
4     int n = first.length, count = 0;
5     for (int i=0; i < n; i++) { // loop from 0 to n-1
6         int total = 0;
7         for (int j=0; j < n; j++) // loop from 0 to n-1
8             for (int k=0; k <= j; k++) // loop from 0 to j
9                 total += first[k];
10        if (second[i] == total) count++;
11    }
12    return count;
13 }

```

(3) BÀI TẬP VỀ NHÀ

Bài tập 1. *Xác định kích thước dữ liệu vào và bậc tăng trưởng độ phức tạp thuật toán.*

Exercise 2.1, Anany's book: Chọn ít nhất 5 trong các bài để thực hiện.

Bài tập 2. *Xác định mối quan hệ độ phức tạp thuật toán theo ký pháp tiệm cận.*

Exercise 2.2, Anany's book: Chọn ít nhất 5 trong các bài để thực hiện.

Bài tập 3. *Thiết kế thuật toán, chứng minh tính đúng và xác định độ phức tạp của thuật toán.*

- Bài toán: Xây dựng thuật toán **sắp xếp chọn** theo ý tưởng sau: sắp xếp n số lưu trong mảng a bằng cách tìm phần tử nhỏ nhất của a và đổi nó với phần tử $a[1]$. Sau đó tìm phần tử nhỏ thứ hai của a , và đổi nó với $a[2]$. Tiếp tục với $n - 1$ phần tử của a .
- Viết mã giả cho thuật toán này.
- Vì sao chỉ cần thực hiện với $n - 1$ phần tử đầu tiên, thay cho cả n phần tử?
- Đưa ra đánh giá thời gian thực hiện thuật toán trong trường hợp tốt nhất và xấu nhất theo kí hiệu O .

- *Viết chương trình và thực hiện với dãy có số lượng lớn phần tử được sinh ngẫu nhiên. Đo thời gian thực hiện, vẽ biểu đồ sự phụ thuộc thời gian vào kích thước n .*
- *(*) Sử dụng bất biến của vòng lặp để chứng minh thuật toán là đúng đắn (chỉ ra bất biến vòng lặp và chứng minh bất biến của vòng lặp thoả mãn đủ ba tính chất khởi tạo, duy trì và kết thúc).*
- *(*) Sinh viên tự đề xuất ít nhất một **thuật toán sắp xếp khác** và thực hiện tương tự các yêu cầu trên.*

(Xem thêm về bất biến vòng lặp trong sách của Rodney R. Howell, “Algorithm: A Top-Down Approach”, Chapter 2 để thực hiện bài tập () nâng cao)*